

Algoritmos Genéticos para Procesamiento de Imágenes

ASCII Art y Compresión por Triángulos Superpuestos

Integrantes: Britez, Santiago; Kantor, Marcos; Sánchez Toledo, Mariano.
Sistemas de Inteligencia Artificial - ITBA

Instituto Tecnológico de Buenos Aires

2026



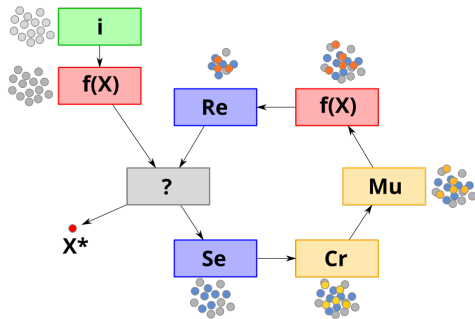
Marco Teórico: Algoritmos Genéticos

Ejercicio 1: ASCII Art con Algoritmos Genéticos

Ejercicio 2: Compresión con Triángulos

Conclusiones

Algoritmo Genético: Ciclo y Componentes



Fuente: Wikimedia Commons

Analogía biológica

Compiten por recursos; los más aptos se reproducen más.

Ciclo evolutivo

1. **Inicialización:** población aleatoria.
2. **Evaluación:** calcular aptitud (fitness).
3. **Selección:** elegir padres por calidad.
4. **Variación:** cruza y mutación.
5. **Reemplazo:** nueva generación.
6. **Parada:** repetir hasta converger.

Componentes clave

- Representación (genotipo)
- Función de aptitud (fitness)
- Operadores: selección, cruza, mutación
- Parámetros: P_c , P_m , tamaño de población, élite

Fundamentos y Ventajas de los AG

Definición (Holland, 1975)

Los algoritmos genéticos (AG) son métodos de optimización estocástica inspirados en la evolución biológica y la selección natural.

Ventajas

- ▶ No requieren derivadas (problemas no lineales).
- ▶ Escapan de óptimos locales.
- ▶ Exploran múltiples soluciones en paralelo.
- ▶ Robustos ante ruido.

Aplicaciones típicas

- ▶ Optimización en espacios complejos.
- ▶ Aprendizaje automático.
- ▶ Diseño de redes neuronales.
- ▶ **Compresión de imágenes** (nuestro caso).

Ejercicio 1 – ASCII Art mediante AG

Problema

Dada una imagen cuadrada en escala de grises, buscar una matriz de $N \times N$ caracteres ASCII cuya representación visual se aproxime lo máximo posible a la imagen original.



Imagen original



Representación ASCII

Idea principal

El problema se transforma en una búsqueda discreta: para cada posición de la matriz debemos elegir **qué carácter representa mejor esa región de la imagen**.

Representación del Individuo

Cromosoma

Un individuo es una cadena de longitud

$$L = N \times N$$

donde cada gen contiene un carácter de un alfabeto ASCII predefinido.

Ejemplo de alfabeto

. : - = + * % @ #

- ▶ Los caracteres presentan distintos niveles de **densidad visual**.
- ▶ Símbolos con poca tinta representan regiones claras; los más densos representan regiones oscuras.
- ▶ La cadena se reorganiza como una matriz $N \times N$ para generar la imagen ASCII.

Espacio de búsqueda discreto: cada gen puede tomar únicamente un carácter del alfabeto.

Función de Aptitud

Evaluación

La matriz de caracteres se renderiza con una fuente monoespaciada y se compara con la imagen objetivo.

Error cuadrático medio

$$E = \frac{1}{WH} \sum_{i=1}^W \sum_{j=1}^H (I(i,j) - R(i,j))^2$$

Fitness

$$\text{fitness} = \frac{1}{1 + E}$$

Menor error $\Rightarrow$ mayor fitness $\Rightarrow$ mejor aproximación visual.

Operadores Genéticos Propuestos

Selección	Cruza	Mutación
▶ Ruleta ▶ Torneo ▶ Ranking ▶ Elitismo Se favorecen individuos con mayor fitness.	▶ Un punto ▶ Dos puntos ▶ Uniforme Los hijos combinan caracteres provenientes de ambos padres.	▶ Reemplazar un carácter ▶ Intercambiar posiciones Introduce nuevos símbolos y mantiene diversidad.

Objetivo

Combinar regiones útiles de distintas soluciones mientras se mantiene suficiente exploración del espacio de búsqueda.

Consideraciones y Posibles Mejoras

Elección del alfabeto

La calidad depende de los caracteres disponibles y de su densidad visual.

. : - = + * % @ # → de menor a mayor densidad

Inicialización

- ▶ Completamente aleatoria.
- ▶ Guiada por el nivel de gris de cada región.

Extensiones

- ▶ Alfabetos más ricos.
- ▶ Métricas perceptuales como SSIM.
- ▶ Evaluación progresiva.
- ▶ Refinamiento local.

Estas estrategias se proponen como posibles configuraciones del diseño conceptual.

Ejercicio 2 – Aproximación con Triángulos

Problema

Dada una imagen objetivo y un número fijo K de triángulos, buscar la configuración de triángulos que minimice el error de reconstrucción.

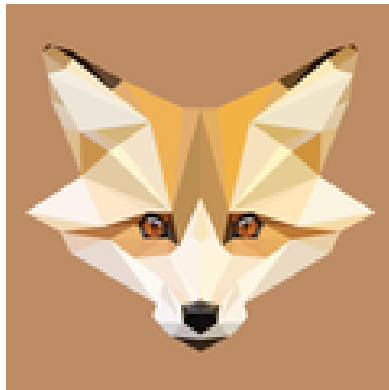
- ▶ Imagen original: 1024×1024 píxeles.
- ▶ Resolución de optimización: 120×120 .
- ▶ Número de triángulos: $K = 600$.
- ▶ Cada triángulo posee geometría, color y transparencia.

Desafío

Cada triángulo tiene 10 parámetros.

$$600 \times 10 = 6000$$

variables continuas a optimizar.



Representación del Individuo

Genotipo

Cada individuo representa una reconstrucción completa formada por K triángulos.

Cada triángulo se codifica utilizando 10 genes:

$$T_i = (x_1, y_1, x_2, y_2, x_3, y_3, r, g, b, \alpha)$$

- ▶ $(x_1, y_1), (x_2, y_2), (x_3, y_3)$: coordenadas normalizadas de los vértices.
- ▶ (r, g, b) : color del triángulo.
- ▶ α : transparencia.

Idea de diseño

La geometría y apariencia de cada triángulo permanecen juntas, permitiendo diseñar operadores genéticos que trabajen sobre **primitivas visuales completas**.

Inicialización completamente aleatoria

Todos los genes se inicializan mediante una distribución uniforme:

$$g \sim U(0, 1)$$

- ▶ La población inicial **no utiliza información de la imagen objetivo**.
- ▶ La configuración inicial depende únicamente de la semilla aleatoria.
- ▶ No se proporciona al AG ninguna aproximación previa de la solución.

Objetivo

Evaluar si la estructura de la imagen puede emerger únicamente mediante selección, cruza y mutación.

Función de Aptitud – Fitness Perceptual

Error a minimizar

$$E = 0.55 E_{\text{color}} + 0.30 E_{\text{bordes}} + 0.15 E_{\text{SSIM}}$$

Color

MSE

Diferencia global entre los colores de ambas imágenes.

Bordes

Sobel

Compara contornos y siluetas importantes.

Estructura

1–SSIM

Evalúa similitud estructural y perceptual.

Interpretación

Un menor valor de E representa una mejor reconstrucción. El AG no busca solamente reproducir colores, sino también **bordes y estructura visual**.

Configuración del Algoritmo Genético

Configuración

- ▶ Población: 64
- ▶ Generaciones: 15 000
- ▶ Elitismo: 3
- ▶ Inyección aleatoria: 2
- ▶ Prob. de cruza: 0.55
- ▶ Prob. de mutación: 0.75

Operadores utilizados

- ▶ Selección: **torneo probabilístico**
- ▶ Cruza: **regional**
- ▶ Mutación: **por triángulo**
- ▶ Supervivencia: **exclusiva + elitismo**

Criterio de diseño

Los operadores fueron elegidos para mantener un equilibrio entre **exploración**, **explotación** y preservación de la estructura geométrica.

Selección – Torneo Probabilístico

Procedimiento

1. Se seleccionan aleatoriamente 4 individuos.
2. Los candidatos se ordenan según su error.
3. El mejor candidato es favorecido con probabilidad $p = 0.8$.

Torneo determinístico

Siempre seleccionaría al mejor candidato.

⇒ Mayor presión selectiva.

Torneo probabilístico

Permite ocasionalmente seleccionar individuos subóptimos.

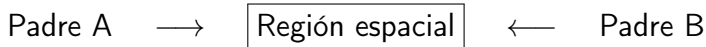
⇒ Mayor diversidad.

Objetivo

Mantener presión selectiva sin eliminar demasiado rápido la diversidad de la población.

Motivación

Una cruce convencional opera sobre posiciones arbitrarias del vector genético. Sin embargo, en este problema los genes representan objetos con **estructura espacial**.



- ▶ Se genera un corte horizontal o vertical sobre la imagen.
- ▶ Se identifican los triángulos pertenecientes a cada región.
- ▶ Se transfieren **triángulos completos** entre los padres.
- ▶ Vértices, color y transparencia permanecen juntos.

Ventaja

El crossover deja de tratar al genoma como una secuencia arbitraria de números y comienza a operar sobre **estructuras visuales**.

Mutación por Triángulo

Mutación estructurada

Cuando se activa la mutación, se seleccionan entre uno y cuatro triángulos y se aplican transformaciones geométricas o visuales.

- ▶ Traslación
- ▶ Escalado
- ▶ Movimiento de vértices
- ▶ Cambio de color
- ▶ Cambio de α
- ▶ Reemplazo del triángulo

Mutación adaptativa

La magnitud de las perturbaciones disminuye progresivamente durante la evolución.

Exploración → Explotación

Al comienzo se permiten cambios importantes. En las últimas generaciones predominan modificaciones pequeñas orientadas al refinamiento.

Supervivencia y Diversidad

Supervivencia exclusiva

La nueva generación se construye principalmente a partir de la descendencia producida durante la generación actual.

Elitismo

Los mejores 3 individuos pasan directamente a la siguiente generación.

Preserva las mejores soluciones

Random injection

Se incorporan 2 individuos completamente aleatorios en cada generación.

Introduce nuevo material genético

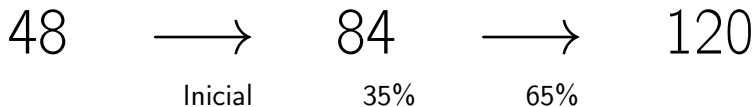
Balance

El elitismo favorece la explotación, mientras que la inyección aleatoria mantiene exploración.

Evaluación Progresiva – Coarse-to-Fine

Problema computacional

En las primeras generaciones las soluciones todavía son muy pobres, por lo que evaluar inmediatamente todos los detalles de una imagen 120×120 resulta innecesario.



- ▶ Primeras generaciones: evaluación rápida a baja resolución.
- ▶ Etapas posteriores: mayor precisión.
- ▶ La resolución completa se reserva para el refinamiento final.

Objetivo

Reducir el costo de la exploración global y concentrar el cómputo en las soluciones más prometedoras.

Aceleración mediante GPU

Principal cuello de botella

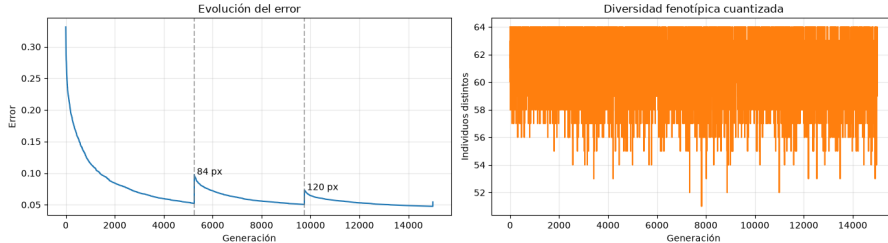
Cada individuo debe ser rasterizado y comparado con la imagen objetivo. Esta evaluación se repite miles de veces durante la evolución.

- ▶ Rasterizado de triángulos ejecutado mediante CUDA.
- ▶ Cálculo del fitness ejecutado en GPU.
- ▶ La lógica evolutiva continúa implementada directamente en Python.
- ▶ Existe fallback automático a CPU.

Ejecución principal

GPU	NVIDIA GeForce RTX 5090
Generaciones	15 000
Población	64
Rendimiento	~ 145 generaciones/s
Tiempo total	103.7 s

Convergencia y Diversidad



- El error disminuye rápidamente durante las primeras generaciones.
- Los saltos observados corresponden a los cambios de resolución:

$$48 \rightarrow 84 \rightarrow 120$$

- Luego de cada cambio, el algoritmo vuelve a refinar la solución.
- La diversidad fenotípica permanece alta durante toda la evolución.

Interpretación

No se observa convergencia prematura significativa de la población.

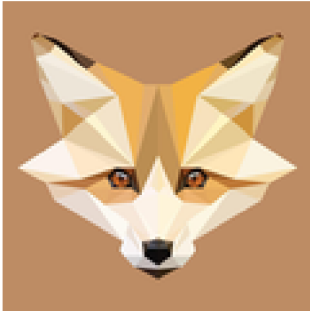
Evolución de la Reconstrucción



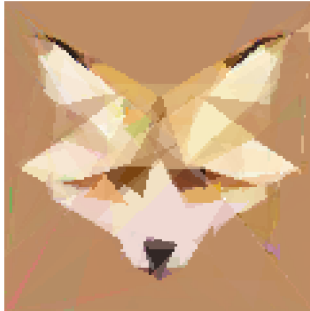
La estructura global emerge rápidamente y luego se refina progresivamente.

Resultado Final

Objetivo



Reconstrucción (error=0.05433)



Alta resolución



600 triángulos



15 000 generaciones



Error final: 0.0543

¿Funciona Solamente para esta Imagen?

Benchmark

Para evaluar robustez y sensibilidad a la semilla se realizaron múltiples ejecuciones independientes.

$$10 \text{ imágenes} \times 3 \text{ semillas} = 30 \text{ ejecuciones}$$

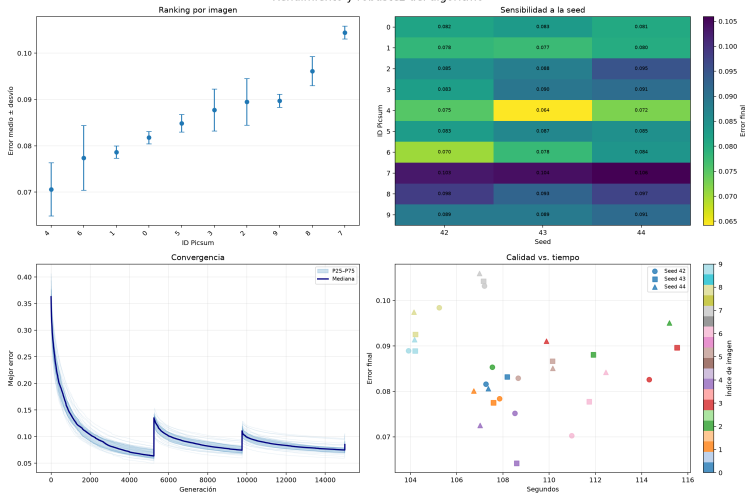
- ▶ Imágenes aleatorias obtenidas desde Picsum.
- ▶ Resolución uniforme de 120×120 .
- ▶ Semillas: 42, 43 y 44.
- ▶ 15 000 generaciones por ejecución.
- ▶ Misma configuración del AG para todos los casos.

Objetivo

Separar la influencia de la **imagen** de la variabilidad introducida por la **semilla aleatoria**.

Resultados del Benchmark

Rendimiento y robustez del algoritmo



La variabilidad entre semillas es menor que la diferencia entre imágenes.

Resultados agregados

- ▶ Error mediano: ≈ 0.0852
- ▶ Peor error: ≈ 0.1059
- ▶ Tiempo mediano: ≈ 103.5 s
- ▶ Ejecuciones exitosas: 100%

Terminación

Las 30 ejecuciones finalizaron por:

`maximum_generations`

Ninguna alcanzó antes el error objetivo ni el criterio de paciencia.

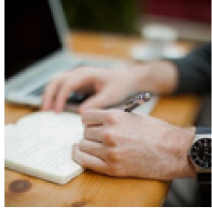
Observación

El comportamiento es estable entre ejecuciones, aunque los criterios de terminación podrían ajustarse, ya que el máximo de generaciones fue siempre el criterio dominante.

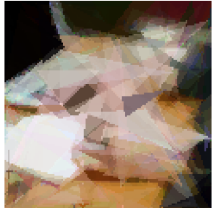
Casos Representativos del Benchmark

Galería representativa: objetivo vs. reconstrucción

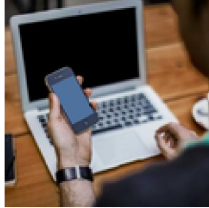
Mejor caso · objetivo
ID 4



Reconstrucción · seed 44
Error=0.0725



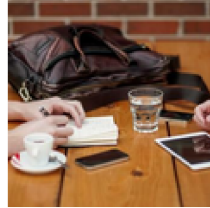
Caso mediano · objetivo
ID 3



Reconstrucción · seed 43
Error=0.0896



Peor caso · objetivo
ID 7



Reconstrucción · seed 43
Error=0.1042



Comparación visual entre el mejor, el caso mediano y el peor caso del benchmark.

Evaluación de todos los métodos

Benchmark

Para evaluar el desempeño de todos los métodos disponibles, se realizó un loop de todos contra todos.

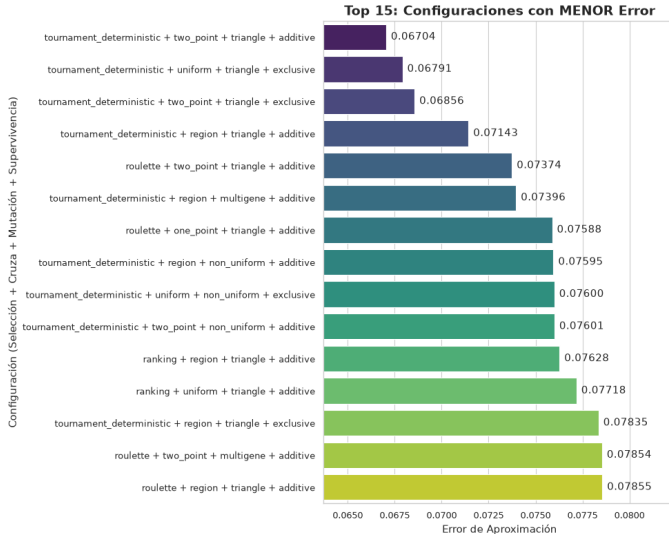
$$3 \text{ Selection method} \times 4 \text{ Crossover method} \times 4 \text{ Mutation method} \times 2 \text{ Survival method} = \boxed{96 \text{ ejecuciones}}$$

- ▶ Patience = 200
- ▶ Generaciones = 5000
- ▶ Semillas: 42, 43 y 44.
- ▶ Initial evaluation size = 32
- ▶ Cómputo en cluster con GPU (Cupy)

Objetivo

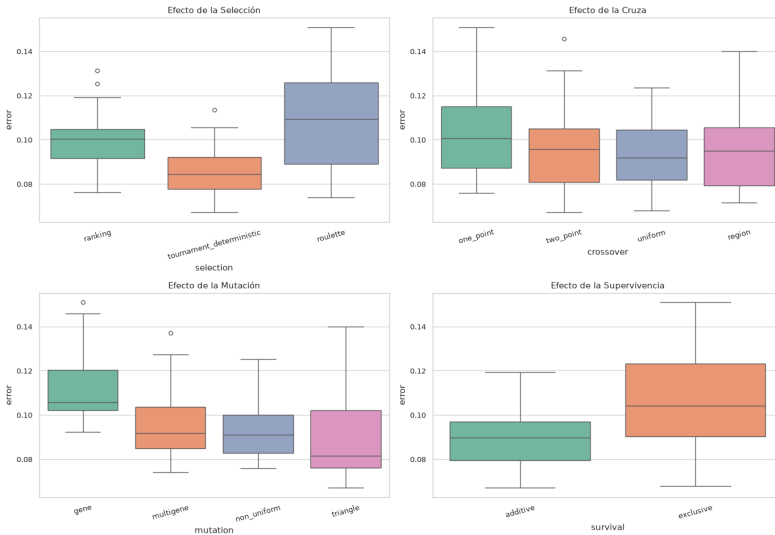
Evaluar como afectan el error de la reconstrucción de la imagen distintas combinaciones posibles de métodos

Resultados de todos los métodos



Resultados de todos los métodos

Distribución del Error por Operador (Menor es mejor)



Conclusiones

- ▶ Se formuló la reconstrucción de imágenes como un problema de optimización continua de **6000 variables**.
- ▶ Representar cada triángulo como una unidad completa permitió diseñar operadores adaptados a la estructura del problema.
- ▶ La **cruza regional** y la **mutación por triángulo** preservan coherencia geométrica durante la evolución.
- ▶ El fitness combinado permite optimizar simultáneamente **color, bordes y estructura**.
- ▶ La evaluación progresiva y CUDA permiten ejecutar 15 000 generaciones en aproximadamente 104 segundos.
- ▶ El benchmark muestra baja sensibilidad a la seed: la dificultad depende principalmente de la imagen objetivo.

Conclusiones sobre todos los métodos evaluados

- ▶ **Superioridad de Selección:** El método de Torneo Determinístico demostró ser **significativamente superior**, concentrando los menores errores de aproximación en comparación con **Ranking** y **Ruleta**.
- ▶ Representar cada triángulo como una unidad completa permitió diseñar operadores adaptados a la estructura del problema.
- ▶ La estrategia de **supervivencia Aditiva** superó consistentemente a la **Exclusiva**.
- ▶ **Compromiso Mutación-Tiempo:** La mutación a nivel **Triangle** logra la mejor fidelidad visual y lidera el top de resultados, pero implica el mayor costo computacional.
- ▶ **Comportamiento de la Cruza:** Los operadores de **cruza Uniforme** y **Región** lograron en promedio los mejores rendimientos globales
- ▶ **Configuración Ganadora:** La configuración élite del experimento combinó **Torneo Determinístico**, **Cruza de Dos Puntos**, **Mutación Triangle** y **Supervivencia Aditiva**, logrando el mínimo error final del conjunto evaluado.

Trabajo Futuro

- ▶ Ajustar los criterios de terminación a partir de los resultados obtenidos en el benchmark.
- ▶ Comparar sistemáticamente los diferentes operadores implementados, modificando un único componente por experimento.
- ▶ Explorar tasas de mutación y crossover adaptativas.
- ▶ Estudiar diferentes cantidades de triángulos K y su relación entre calidad de reconstrucción y complejidad.
- ▶ Extender el enfoque a imágenes de mayor resolución.
- ▶ Incorporar estrategias adicionales de refinamiento local sobre las mejores soluciones.

Idea principal

En problemas de alta dimensionalidad no alcanza con elegir un algoritmo genético: los operadores y la estrategia de evaluación deben **explotar la estructura específica del problema**.

¡Gracias!

